

GitHub公開時の注意点とバイナリ配布

<https://gemini.google.com/app/bcbf70b7acdf3321>

User prompt: AIでアプリを開発している、作成したものはGITHUBにPRIVATEで管理している、これをPUBRICにするとときに注意すべき事項とバイナリのEXEとDLL等一式を公開したい、適切なサイトと注意事項を教えて

Response: **GitHubリポジトリをPublicにする前の注意点**

- **コミット履歴のシークレット混入チェック:** 過去の全コミットログにAPIキー、トークン、パスワード、絶対パス（個人名を含むローカルパス）が含まれていないか確認します。一度でもpushした履歴は公開と同時に閲覧・自動スキャンされるため、履歴にある場合はgit-filter-repoやBFG Repo-Cleanerで完全に消去するか、新規リポジトリを作成して最新コードのみを移行します。
- **環境変数ファイルの除外:** .envや認証情報ファイルを.gitignoreに明記し、リモートに上がっていないことを確認します。
- **ライセンスの明記:** LICENSEファイルを配置します（例: 商用利用や改変を広く許可するならMIT、派生コードの公開を義務付けるならGPLv3など）。ライセンスがない場合、デフォルトで著作権が完全に保持され他人が利用できなくなります。
- **README.mdの整備:** アプリの概要、動作環境（OS / ランタイム）、インストール・実行手順、ライセンス、免責事項を記述します。
- **著作権・サードパーティ製ライブラリの規約:** 利用している外部ライブラリやAIモデル、モデルの生成物に関するライセンス要件（クレジット表記義務など）を満たしているか確認します。

バイナリ（EXE / DLL一式）の公開に適したサイト

プラットフォーム	特徴・用途	おすすめ度
GitHub Releases	リポジトリ画面の「Releases」からZIP等を直接添付して配布可能。ソースコードと同一場所でバージョン管理できる。	最も推奨
Vector / 窓の杜	国内のWindows向け定番ソフト登録サイト。日本の一般ユーザーへの露出・信頼性向上に向く。	国内向けに最適
itch.io / BOOTH	ゲーム・ツール・自作アプリの配布や有料販売が可能なプラットフォーム。	クリエイター向け

バイナリ公開時の注意点

- **Git本体にバイナリをコミットしない:** .exe や .dll などの大容量バイナリはリポジトリの履歴を肥大化させるため、リポジトリ内ではなく **GitHub Releases** のアセット（ZIPファイル等）としてアップロードします。
 - **Windows SmartScreen / ウィルス誤検知対策:**
 - コード署名証明書（Code Signing Certificate）がない個人製バイナリは、Windows Defender SmartScreen で「保護されたPC」として警告が出やすくなります。
 - 事前に VirusTotal 等でスキャンして無害であることを確認し、READMEに「署名がないため警告が出る場合の対処法」やハッシュ値（SHA-256）を明記しておくで利用者の安心感に繋がります。
 - **依存DLL・ランタイムの同梱:** 実行端末に VC++ 再頒布可能パッケージや .NET ランタイム等が必要な場合、インストーラー（Inno Setup 等）でまとめるか、自己完結型（Self-contained）ビルドにして配布します。
 - **免責事項の記載:** 「本ソフトウェアの使用によって生じた損害について作者は責任を負いません」といった免責事項を README および配布ファイル内のテキストに明記します。
-

User prompt: コード署名証明書（Code Signing Certificate）を取得する手段は？

Response: Windowsアプリ用のコード署名証明書（Code Signing Certificate）を取得・利用する主な手段は以下の通りです。

証明書の種類

- **OV（Organization Validation / 組織認証）:** 組織（企業・個人事業主）向けの実在証明付き証明書。
- **EV（Extended Validation / 拡張認証）:** より厳格な審査を経て発行される証明書。取得直後から Windows SmartScreen の警告を回避しやすい特徴があります。
- **個人の場合:** 近年は認証局（CA）のセキュリティ基準改定により、個人名義での取得審査が厳格化しています。個人事業主としての屋号登録や公的書類の提出が求められるケースが一般的です。

取得・利用の主な手段

- **認証局（CA）または正規代理店から購入する:**
 - **主な提供元:** DigiCert、Sectigo、GlobalSign など。
 - **国内代理店:** GMOグローバルサイン、デジサート・ジャパン、SSLストア等。

- **現在の仕様（ハードウェア要件）**：現在の業界基準では、秘密鍵をクラウドHSM（ハードウェア・セキュリティ・モジュール）または物理USBトークン（FIPS準拠）に格納して署名する必要があります（従来のファイル形式 .pfx のみでの直接配布は廃止されています）。
- **費用目安**: 年間数万円～十数万円程度。
- **クラウド署名サービスを利用する**:
 - **Azure Trusted Signing（旧 Microsoft Azure Code Signing）**：Microsoft が提供するクラウドベースの署名サービス。物理トークンが不要で、CI/CDパイプライン（GitHub Actions 等）との連携が容易かつ比較的安価に利用可能です。
 - **DigiCert ONE / Keylocker 等**: クラウド上でキー管理と署名を完結できるエンタープライズ向けサービス。
- **自己署名証明書（オレオレ証明書）を作成する（※無料・開発用）**:
 - PowerShellの `New-SelfSignedCertificate` や `openssl` で自作可能。
 - **注意点**: 自身のPC（証明書を信頼済みルートに手動インストールした環境）以外では警告が出るため、一般配布用のSmartScreen回避には使えません。
- **Microsoft Store経由で配布する（代替案）**:
 - 証明書を個別に購入せず、アプリをパッケージ化（MSIX等）して Microsoft Store に登録・審査を通す方法。
 - Microsoft側で署名が付与されるため、個別のコード署名証明書を購入するコストを抑えつつ安全に配布できます（開発者アカウント登録料は数千円程度で一度きり）。

User prompt: 個人だと実質どれがいいの？

Response: 個人開発の場合、「署名なし+READMEで回避方法を案内する」か「Microsoft Storeで配布する」の2択が実質的な最適解です。

通常のコード署名証明書は、審査基準改定により個人名義での購入が極めて困難（または不可）になっており、購入できても年間数万～十数万円と物理トークン維持のコストがかかります。

個人の現実的な選択肢

- **1. 署名なし + GitHub Releasesで無料配布（一番多い選択肢）**
 - **費用**: 0円

- **対応策:** SmartScreenの青い警告画面が出た際の「詳細情報」→「実行」という起動手順をREADMEにスクショ付きで明記します。また、VirusTotalのスキャン結果リンクやSHA-256ハッシュ値を添えて安全性をアピールします。
- **ダウンロード実績による緩和:** 配布実績やダウンロード数が増えると、Microsoftの評判システム（SmartScreen Reputation）により徐々に警告が出にくくなります。
- **2. Microsoft Store 経由で配布（信頼性を重視する場合）**
 - **費用:** 個人アカウント登録料（初回約19ドルのみ、買い切り）
 - **メリット:** Microsoft側で安全性が審査・署名されるため、SmartScreen警告が出ず、ユーザーもワンクリックで安全にインストール・更新できます。
 - **デメリット:** アプリのパッケージ化（MSIX化）が必要な点と、Storeのガイドライン審査があります。

結論としての目安

- **まずは手軽にGitHubで公開したい場合:** 証明書は買わず、署名なし+丁寧なREADME案内で十分です。
- **警告を絶対に消して一般ユーザーに使ってもらいたい場合:** 証明書を年額で買うのではなく、**Microsoft Storeへの登録を検討するのが最も現実的で経済的**です。

User prompt: GITHUBのPUBRIC設定はどこにあるの？

Response: GitHubリポジトリを **Private（非公開）** から **Public（公開）** に変更する手順は以下の通りです。

リポジトリの公開手順

1. 対象のリポジトリを開く

- GitHubにログインし、公開したいリポジトリのトップページを開きます。

2. 「Settings」タブを開く

- 画面上部にあるメニューバー（「Code」「Issues」「Pull requests」など）の右端にある「Settings」（歯車アイコン）をクリックします。

3. 最下部の「Danger Zone」までスクロールする

- 「General」設定画面が開きますので、ページを一番下までスクロールします。

- 赤枠で囲まれた「**Danger Zone**」というエリアがあります。

4. 「**Change visibility**」をクリック

- Danger Zone 内の「**Change repository visibility**」（または「**Change visibility**」）という項目にあるボタンをクリックします。

5. 「**Change to public**」を選択して確定する

- ドロップダウンまたはポップアップが表示されるので、「**Change to public**」を選択します。
- 警告内容（誰でもコードを閲覧・クローンできるようになる等）を確認する画面が出ます。
- 指定されたテキスト（例: リポジトリ名や確認メッセージ）を入力し、「**I understand, change repository visibility**」をクリックして完了です。

User prompt: GITHUBの中で他のユーザーとコミュニケーション取れることはできるの？

Response: GitHub上には、他のユーザーとコミュニケーションを取るための機能が複数用意されています。

用途に応じて以下のような手段を使い分けるのが一般的です。

GitHub内の主なコミュニケーション機能

- **Issues（課題・要望管理）**

- **用途:** バグ報告、新機能のリクエスト、タスクの相談など。
- スレッド形式でコメントのやり取りや画像の添付、進捗ステータス（Open / Closed）の管理ができます。

- **GitHub Discussions（掲示板・コミュニティ機能）**

- **用途:** Q&A、アイデアのブレインストーミング、雑談など。
- Issuesほど形式張らず、フォーラム感覚でユーザー同士が質問や意見交換を行えます（リポジトリの「Settings」から有効化可能）。

- **Pull Requests（PR / コード提案へのレビュー）**

- **用途:** 他の開発者から送られてきたコード変更に対するレビューやフィードバック。
- 特定の行を指定してコメントや修正提案を行えます。

- **メンション機能 (@ユーザー名)**

- Issue、PR、コミットコメントなどで @username を記述すると、相手に直接通知 (Notification / メール) が届きます。

- **GitHub Profiles / 連絡先**

- ユーザーのプロフィール画面に記載されているSNSリンク (X/Twitter等) や公開メールアドレスを通じて連絡を取ることも可能です。
- ※GitHubには一般的な「1対1のダイレクトメッセージ (DM) 機能」は直接備わっていないため、非公開のやり取りが必要な場合はプロフィール記載のメールやSNSを利用します。

User prompt: ユーザー名には@をつけるのが普通？

Response: GitHubでの使い方や文脈によって異なります。

- **相手を呼び出す (メンション・通知を送る) とき: @ をつけます。**

- Issue、Pull Request、Discussions のコメント欄などで @username と書くと、その相手に通知が届き、ユーザーページへのリンクになります。

- **アカウント作成・ログイン時: @ はつけません。**

- 登録するユーザー名 (Username) 自体には @ 記号は含めず、半角英数字とハイフン (-) のみで設定します。

- **URLで表記するとき: @ はつけません。**

- プロフィールURLは [https://github.com/username] (https://github.com/username) のようになります。

テキスト上で「特定のユーザーを指名して話しかける・言及する」場合は、@ユーザー名 と表記するのが一般的です。

User prompt: ディスクドライブの速度を図るアプリを作った、名前はSSD-Speedというんだけど英語圏も含めてもっといい名前ある？既存のアプリと被らない様に

Response: SSD-Speed は機能が伝わりやすい反面、一般的すぎて既存のベンチマークソフト (CrystalDiskMark, ATTO Disk Benchmark, AS SSD Benchmark, Blackmagic Disk Speed Test など) やWeb上の検索結果に埋もれやすい傾向があります。

海外ユーザーにも分かりやすく、GitHubや検索で被りにくいネーミングの方向性・候補をいくつか挙げます。

1. 機能直球型（検索で見つかりやすく、シンプル）

- **DrivePulse**（ドライブの状態・脈動を測るイメージ）
- **DiskSprint**（ドライブの瞬間的な高速測定を想起させる）
- **DriveGauge**（ゲージのようにドライブの速度を可視化する）
- **IOFlash**（SSDの高速なI/O速度を表現）

2. 軽量・俊敏さを強調するタイプ（個人開発・ポータブル系におすすめ）

- **DiskBrisk**（Brisk: 軽快な、きびきびした）
- **SSDSnappy**（Snappy: キビキビ動く、手軽な測定感）
- **DriveSwift**（Swift: 迅速な、素早い）
- **SSDTempo**（SSDのスピード・リズムを測る）

3. クール・モダン系（GitHub映えするブランド名）

- **VeloDisk**（Velocity: 速度 + Disk）
- **DiskSpectra**（SSDの読み書き性能をスペクトル分析するイメージ）
- **StrataBench**（SSDのレイヤー・構造を測るベンチマーク）

名前を決定する前のチェックポイント

- **GitHubの検索:** GitHubの検索窓で候補名を検索し、同名のリポジトリが存在しないか確認します。
- **Google検索:** 既存の商標や有名ソフトウェアと完全に衝突していないか確認します。
- **リポジトリ名・実行ファイル名の揃えやすさ:** DrivePulse.exeのように、バイナリ名やリポジトリ名（例: username/drive-pulse）にしたときの収まりが良いものを選ぶのがおすすめです。

User prompt: 特徴としてはCrystalDiskMarkとほぼ同じ機能だけどより高速（数分の1の時間）で測定結果が出せる、またCrystalにはないSSD本体のキャッシュを無効化してドライブの素の速度も計測できるんだ、ぴったりの名前を考えて

Response: 「短時間での計測 (Rapid / Flash / Snap)」と「キャッシュ無効化による素の性能測定 (Raw / Direct / Pure / Unbuffered)」という強力な強みを前面に出したネーミング案です。

1. 「素の速度 (キャッシュ無効)」を強調するタイプ (技術者・エンスージアスト向け)

- **RawDiskMark** (Raw: 生の、加工なし + Benchmark。素の性能を測るツールとして最も直感的)
- **DirectDriveMark** (Direct: キャッシュをバイパスして直で叩くイメージ)
- **PureDiskBench** (Pure: キャッシュ等のごまかしがない純粋な性能を測定)
- **NakedDisk / DiskNaked** (SSDの「ありのまま」の素の速度を暴くイメージ)

2. 「圧倒的な測定スピード (短時間)」を強調するタイプ

- **QuickDiskMark** (CrystalDiskMarkを意識しつつ、短時間で終わる利便性を訴求)
- **SnapBench / DiskSnap** (Snap: 指を鳴らす一瞬で終わるベンチマーク)
- **RapidDiskBench** (Rapid: 高速・迅速な測定)
- **FlashMark** (SSDのフラッシュメモリ + 閃光のように速い測定のダブルミーニング)

3. 「高速 + 素の測定」の両方を兼ね備えたモダン系 (GitHub映え)

- **RawSprint** (素の性能 × 瞬発的な短時間測定)
- **TrueSpeed Disk** (キャッシュに惑わされない真の速度)
- **FastRawBench** (速く、生のデータを出すツール)
- **DirectPulse** (ダイレクトアクセス × 短時間の状態測定)

おすすめの組み合わせ例

- 直感的な分かりやすさ重視: **RawDiskMark** (READMEで “Fast benchmark with Raw-NAND / direct I/O mode” と説明しやすい)
- 短時間 & モダンな響き重視: **DiskSnap** または **RawSprint**

User prompt: CrystalDiskMark互換の測定ができるって表示してもいいのかな? 条件だけでコードとかUIは流用していない

Response: コードやUIを流用せず、測定条件 (テストサイズ、スレッド数、キュー数、ブロックサイズ、ランダム/シーケンシャル設定など) を合わせただけであれば、**「CrystalDiskMark**

互換の設定/プロファイルで測定可能」と表記すること自体は法的に問題ありません。

ただし、商標や誤認トラブルを避けるために以下の点に配慮した書き方にする必要があります。

表記時の重要なポイント

- 「公式」や「派生」と誤解させない表現にする:
 -  *CrystalDiskMark Fast Edition, CrystalDiskMark 互換ソフト*（アプリ名や見出しに使うと商標の混同を招く恐れがあります）
 -  “*CrystalDiskMark-equivalent test settings*”（CrystalDiskMarkと同等のテスト設定）
 -  “*Supports test profiles matching CrystalDiskMark default parameters (e.g., SEQ1M Q8T1, RND4K Q32T1)*”
- 免責事項・クレジットをREADMEに明記する:
 - READMEの末尾などに「CrystalDiskMarkはhiyohiyo氏の商標/著作物であり、本アプリは非公式の独自実装であること」を記載しておくトラブルを未然に防げます。
 - 記載例:

Disclaimer: CrystalDiskMark is a registered trademark or software by hiyohiyo / Crystal Dew World. This software is an independent implementation and is not affiliated with or endorsed by Crystal Dew World.
- 測定値の差異についての注意書き:
 - 測定時間が短縮されていたり、キャッシュバイパスなどの独自処理が入っていたりする場合、完全に同一の条件でもCrystalDiskMark本家と若干の数値差が出る場合があります。「設定プロファイルは同等だが、内部実装の違いによりスコアが完全一致しない場合がある」旨を注記しておく、ユーザーからのバグ報告を回避できます。

User prompt: QuickDiskMarkだとなんかパクった感が出ちゃうな、素直にQuickDiskBench、これどう？イメージしやすい？

Response: QuickDiskBench は非常に分かりやすく、素晴らしいネーミングです。

評価とメリット

- **パクリ感の解消:** 「~Mark」ではなく「Bench (Benchmarkの略)」を使うことで、CrystalDiskMarkの単なるクローン・偽物感をうまく消しつつ、自作の独立したツールとしての風格が出ます。
- **直感的なわかりやすさ:** 英語圏でも国内でも「ディスクのベンチマークソフトであり、測定が速い (Quick)」という特徴が1秒で伝わります。
- **短縮名・リポジトリ名の収まりが良い:**
 - リポジトリ名: quick-disk-bench や QuickDiskBench
 - 実行ファイル名: QuickDiskBench.exe
 - 略称: QDB (短縮アイコンやログでも使いやすい)

READMEでのアピール文 (例)

アプリ名が「Quick (短時間)」に寄っている分、もう1つの強力な強みである「キャッシュ無効化 (素の速度)」をREADMEのサブタイトルやキャッチコピーで補うと完璧です。

QuickDiskBench

A blazingly fast disk benchmark tool with raw performance (direct I/O / cache-bypass) measurement.

User prompt: 同じ名前のアプリは世の中にないないよね？

Response: QuickDiskBench という名前の有名な既存アプリや主要な商標ソフトウェアはありません。

同名・類似のチェック状況

- **主要ベンチマークソフトとの重複:**
 - 既存の著名ツールはCrystalDiskMark、AS SSD Benchmark、ATTO Disk Benchmark、Anvil's Storage Utilities、Blackmagic Disk Speed Test などであり、QuickDiskBench という有名アプリは存在しません。
- **GitHub等の命名傾向:**
 - 個人開発の小規模リポジトリやテスト用コードで似た名前 (quick-disk-test など) が使われることはあっても、代表的なプロダクトとして定着しているものはありません。
- **商標・名称としての安全性:**

- 一般名詞（Quick / Disk / Bench）を組み合わせた造語であるため、他社の独自ブランド商標を直接侵害するリスクも極めて低いです。

公開前の最終セルフチェック

1. **GitHub検索:** [<https://github.com/search?q=QuickDiskBench>]
(<https://github.com/search?q=QuickDiskBench>) で検索し、同名のリポジトリが存在しないか念のため確認する。
2. **リポジトリ名の確保:** 空いていれば、GitHubで QuickDiskBench または quick-disk-bench としてリポジトリを作成して名前を確保する。

独自ツールとして名乗るのに全く問題のない、クリーンで分かりやすい名前です。
